

hello.py

```
1 # Python From Zero
2 name = "future coder"
3 print(f"Hello, {name}!")
4
```

OUTPUT

Hello, future coder!

Python From Zero

Learn it. Type it. Build it.

Build a strong Python foundation through simple concepts, practical examples, and projects.

Your roadmap, **set up.**

Install once, then learn by doing. No experience needed.

YOUR ROADMAP

Your First Program

- 01** Print text, store a value and add comments.

Variables and Data

- 02** **Types**
Store information and use it later.

Making Decisions

- 03** Run different code depending on a condition.

Loops

- 04** Repeat code without writing it again and again.

Lists and Dictionaries

- 05** Store many values in one variable.

Functions

- 06** Write code once and reuse it.

Errors and Files

- 07** Handle problems and save data.

Project: Number Guessing Game

- 08** Put it all together.

Project: To-Do List App

- 09** Build a real tool.

SET UP IN 5 MINUTES

- 1** Download Python 3 from `python.org` and install it.
- 2** Open a terminal and type `python --version`. On some systems, use `python3`.
- 3** Pick an editor: IDLE (included), Thonny or VS Code.
- 4** Create `hello.py`, write `print("Hi")` and run `python hello.py`.

HOW TO LEARN FROM THIS GUIDE

Read

the idea

Type

don't copy

Run

see the output

Change

experiment

Break

then fix it

Your First Program

Print text, store a value and add comments.

THE IDEA

A program is a list of instructions. Python reads it from top to bottom, one line at a time.

KEY POINTS

- `print()` shows text on the screen.
- Text goes inside quotes.
- `#` starts a comment. Python ignores it.
- Run your file with `python hello.py`.

hello.py

```
1 # My first program
2 print("Hello, world!")
3 name = "Sam"
4 print("Welcome,", name)
5 print(2 + 3)
6
```

OUTPUT

```
Hello, world!
Welcome, Sam
5
```

COMMON MISTAKE

Forgetting a quote or bracket, like `print("Hello`. Python shows a `SyntaxError` and points to the line.

TRY IT YOURSELF

- ☐ Print your own name.
- ☐ Print the result of 12×7 .
- ☐ Add a comment above every line.

QUICK CHECK

What does `print(2 + 3)` show? Answer: 5, because Python adds the numbers first.

Variables and Data Types

Store information and use it later.

THE IDEA

A variable is a labelled box that holds a value. Use `=` to store a value in it.

KEY POINTS

- Four basic types: `int`, `float`, `str`, `bool`.
- Names use letters, numbers and `_`, and can't start with a number.
- An f-string, like `f"Hi {name}"`, inserts values into text.
- `input()` always returns text. Use `int()` to convert.

types.py

```
1 name = "Sam"
2 age = 20
3 height = 1.72
4 is_student = True
5 print(type(age))
6 print(f"{name} is {age} years old")
7 print(age + 5)
8
```

OUTPUT

```
<class 'int'>
Sam is 20 years old
25
```

COMMON MISTAKE

Adding text and a number, like `"Age: " + 20`, gives a `TypeError`. Use an f-string instead.

TRY IT YOURSELF

- ☐ Store your name, age and a favourite number.
- ☐ Print one sentence using an f-string.
- ☐ Print your age in 10 years.

QUICK CHECK

Which type is 3.5? Answer: float, a number with a decimal point.

Making Decisions

Run different code depending on a condition.

THE IDEA

Programs make choices with `if`. Python checks a condition that is either `True` or `False`.

KEY POINTS

- Compare with `==`, `!=`, `<`, `>`, `<=`, `>=`.
- Combine conditions with `and`, `or`, `not`.
- Indent with 4 spaces to show what is inside the `if`.
- `=` stores a value. `==` compares two values.

grade.py

```
1 score = 72
2 if score >= 90:
3     print("Excellent")
4 elif score >= 60:
5     print("Good job")
6 else:
7     print("Keep practising")
8
```

OUTPUT

Good job

COMMON MISTAKE

Writing `if x = 5` instead of `if x == 5`, or forgetting the colon at the end of the line.

TRY IT YOURSELF

- ☐ Ask for an age and print "adult" or "minor".
- ☐ Check if a number is even using `%`.
- ☐ Build a simple grade checker.

QUICK CHECK

What does `7 % 2` give? Answer: 1, the remainder. So 7 is odd.

Loops

Repeat code without writing it again and again.

THE IDEA

A loop repeats a block of code. Use `for` to repeat a set number of times, and `while` to repeat until something changes.

KEY POINTS

- `range(1, 4)` gives 1, 2, 3. The end value is not included.
- `while` repeats as long as its condition is True.
- `break` exits a loop. `continue` skips to the next round.
- Press Ctrl+C to stop a loop that never ends.

loops.py

```
1 for i in range(1, 4):
2     print("Round", i)
3
4 count = 3
5 while count > 0:
6     print(count)
7     count -= 1
8 print("Go!")
9
```

OUTPUT

```
Round 1
Round 2
Round 3
3
2
1
Go!
```

COMMON MISTAKE

A while loop that never changes its condition runs forever. Make sure something inside the loop changes.

TRY IT YOURSELF

- ☐ Print the numbers 1 to 10.
- ☐ Print the 5 times table.
- ☐ Add up the numbers from 1 to 100.

QUICK CHECK

How many times does `range(3)` repeat? Answer: 3 times, with values 0, 1 and 2.

Lists and Dictionaries

Store many values in one variable.

THE IDEA

A list keeps items in order. A dictionary stores pairs: a key and its value.

KEY POINTS

- List positions start at 0, so the first item is `items[0]`.
- Use `append()` to add and `len()` to count.
- Look up a dictionary value with its key: `person["name"]`.
- Loop over both with `.items()`.

collections.py

```
1 fruits = ["apple", "banana"]
2 fruits.append("mango")
3 print(len(fruits), fruits[0])
4
5 person = {"name": "Sam", "age": 20}
6 person["city"] = "Paris"
7 for key, value in person.items():
8     print(key, value)
9
```

OUTPUT

```
3 apple
name Sam
age 20
city Paris
```

COMMON MISTAKE

Asking for `fruits[3]` in a 3-item list causes an `IndexError`. The last position is 2.

TRY IT YOURSELF

- ☐ Make a list of 5 favourite movies and print each.
- ☐ Create a dictionary of 3 contacts.
- ☐ Find the longest word in a list.

QUICK CHECK

If `items = ['a', 'b', 'c']`, what is `items[1]`? Answer: 'b', because counting starts at 0.

Functions

Write code once and reuse it.

THE IDEA

A function is a named block of code you can run whenever you need it, as many times as you like.

KEY POINTS

- Define with `def` `name(parameters):`.
- `return` sends a result back to the caller.
- Parameters can have default values.
- Keep each function focused on one job.

functions.py

```
1 def greet(name, greeting="Hello"):
2     return f"{greeting}, {name}!"
3
4 def add(a, b):
5     return a + b
6
7 print(greet("Sam"))
8 print(add(4, 6))
9
```

OUTPUT

```
Hello, Sam!
10
```

COMMON MISTAKE

Using `print` inside a function when you need `return`. Print shows a value, return lets you use it later.

TRY IT YOURSELF

- ☐ Write `square(n)`.
- ☐ Write `is_even(n)` that returns True or False.
- ☐ Write a function that returns a name in capitals.

QUICK CHECK

What does return do? Answer: Sends a value back to the code that called the function.

Errors and Files

Handle problems and save data.

THE IDEA

Errors will happen, and that is normal. Python tells you what went wrong and on which line.

KEY POINTS

- Read the last line of an error message first.
- `try / except` catches an error so your program keeps going.
- `with open()` opens and closes files safely.
- Modes: `"r"` read, `"w"` write, `"a"` append.

```
files.py
1 try:
2     number = int("42a")
3 except ValueError:
4     print("That is not a number")
5
6 with open("notes.txt", "w") as f:
7     f.write("Learn Python\n")
8 with open("notes.txt") as f:
9     print(f.read())
10
```

OUTPUT

```
That is not a number
Learn Python
```

COMMON MISTAKE

Opening a file with `"w"` erases what was there. Use `"a"` to add to the end.

TRY IT YOURSELF

- ☐ Ask for a number and handle bad input.
- ☐ Save 3 lines to a file, then read them back.
- ☐ Append one more line without erasing.

QUICK CHECK

Which file mode adds text without erasing? Answer: `"a"` (append).

Number Guessing Game

The computer picks a number. You guess it.

random

while loop

input()

if / elif / else

guess.py

```
1 import random
2
3 secret = random.randint(1, 20)
4 tries = 0
5
6 while True:
7     guess = int(input("Guess 1-20: "))
8     tries += 1
9     if guess < secret:
10         print("Too low")
11     elif guess > secret:
12         print("Too high")
13     else:
14         print(f"Correct in {tries} tries!")
15         break
16
```

OUTPUT

```
Guess 1-20: 10
Too low
Guess 1-20: 15
Too high
Guess 1-20: 13
Correct in 3 tries!
```

HOW IT WORKS

- 1 `random.randint(1, 20)` picks the secret number.
- 2 `while True` keeps asking until you win.
- 3 `int(input())` turns your typing into a number.
- 4 `break` ends the loop when you guess right.

UPGRADE IT

- ☐ Limit the player to 5 tries.
- ☐ Handle bad input with `try / except`.
- ☐ Ask to play again.

To-Do List App

Add and view tasks from a simple menu.

list

while loop

enumerate()

menu with input()

 todo.py

```

1 tasks = []
2
3 while True:
4     choice = input("add / show / quit: ")
5     if choice == "add":
6         tasks.append(input("Task: "))
7     elif choice == "show":
8         for i, t in enumerate(tasks, 1):
9             print(i, t)
10    elif choice == "quit":
11        break
12 print("Bye!")
13

```

OUTPUT

```

add / show / quit: add
Task: Buy milk
add / show / quit: add
Task: Study Python
add / show / quit: show
1 Buy milk
2 Study Python
add / show / quit: quit
Bye!

```

HOW IT WORKS

- 1 `tasks = []` starts with an empty list.
- 2 The loop shows the menu again and again.
- 3 `append()` adds a new task.
- 4 `enumerate(tasks, 1)` numbers the tasks from 1.

UPGRADE IT

- ☐ Add a "done" option to remove a task.
- ☐ Save tasks to a file.
- ☐ Handle unknown commands.

Keep this handy.

The syntax you have learned, on one page.

Show text	<code>print("Hi")</code>
Get input	<code>name = input("Name: ")</code>
Convert	<code>int("5") float("2.5")</code>
Check type	<code>type(x)</code>
Count items	<code>len(items)</code>
Decide	<code>if x > 0: ... else:</code>
Repeat N times	<code>for i in range(5):</code>
Repeat while	<code>while condition:</code>
Add to list	<code>items.append(x)</code>
Dictionary	<code>d["key"]</code>
Function	<code>def add(a, b): return a + b</code>
Catch errors	<code>try: ... except ValueError:</code>
Read a file	<code>with open("f.txt") as f:</code>

NEXT STEPS

- ☐ Practise 20 minutes every day
- ☐ Build one small project a week
- ☐ Learn classes (OOP) and modules
- ☐ Try libraries: requests, pandas, Flask
- ☐ Save your projects on GitHub

MORE PROJECT IDEAS

[Calculator](#)[Quiz game](#)[Password generator](#)[Unit converter](#)[Expense tracker](#)[Hangman](#)

You can code. Keep building.